Java Basics

A complete beginner guide: OOP Concepts & Introduction to Java

■ Table of Contents

1. What is Programming? Procedural vs OOP

Before diving into Java, it helps to understand two different **styles of programming** — because Java is built on the second one.

> ■ **Real-World Analogy**
>
> Think of a factory. In a **procedural** factory, workers follow a fixed list of instructions from top to bottom — step 1, step 2, step 3. In an **OOP** factory, everything is organized into departments (objects) — each department knows its own job and has its own data and tools.

Procedural Programming

- Focuses on a **sequence of instructions** to solve a problem — like a recipe.
- Uses a **top-down** approach: break a big problem into smaller functions.
- Data is **separate** from the functions that work on it — anyone can access it.
- Examples: C, Pascal.

Object Oriented Programming (OOP)

- Combines **data + functions** into a single unit called an **object**.
- Uses a **bottom-up** approach: build small objects, then combine them.
- Data is **hidden** inside objects — only accessible through defined methods.
- Examples: Java, C++, Python, C#.

Side-by-Side Comparison

Procedural Programming	Object Oriented Programming
Programs divided into functions	Programs divided into objects
Data is not hidden — open to all	Data is hidden inside objects
Top-down design approach	Bottom-up design approach
Data shared through functions	Objects communicate through methods
Focus is on procedure (HOW)	Focus is on data (WHAT)
Harder to manage large programs	Easier to scale and maintain

2. Core OOP Concepts Explained

Java is built on these 8 core OOP concepts. Understanding them is essential before writing any Java code.

① Objects

An **object** is the basic building block of any OOP program. It is a real-world entity that has **data (attributes)** and **behaviour (methods)**.

- Every object has a unique identity (name).
- Every object belongs to a class.
- Example: *apple*, *mango*, *orange* are objects of the class *Fruit*.

② Class

A **class** is a blueprint or template from which objects are created. It defines what data and methods all objects of that type will have.

- A class is a user-defined data type.
- One class can produce unlimited objects.
- Example: *Planets*, *Sun*, *Moon* are objects of class *SolarSystem*.

③ Data Encapsulation & Data Hiding

Encapsulation means wrapping data and the methods that work on that data together inside a single unit (class).

- **Data hiding** is a result of encapsulation — the internal data of an object is hidden from the outside world.
- Outside code cannot directly touch the data — it must go through methods.

④ Data Abstraction

Abstraction means showing only the essential features and hiding the background complexity. When you drive a car, you use the steering wheel — you don't need to know how the engine works.

- Separates *what* an object does from *how* it does it.

⑤ Inheritance

Inheritance lets a new class (child/subclass) acquire the properties and methods of an existing class (parent/superclass). This enables **code reuse**.

- The original class is called the **base class** or **superclass**.
- The new class is called the **derived class** or **subclass**.
- Example: *Dog* inherits from *Animal* — it gets all Animal properties, plus its own.

⑥ Overloading

Overloading allows a method or operator to have **multiple meanings** depending on context.

- **Method overloading:** same method name, different parameters. E.g., `add(int a, int b)` and `add(double a, double b)`.
- **Operator overloading:** reusing an operator (like +) for new data types.

⑦ Polymorphism

Polymorphism means 'many forms'. A single method or operator can behave differently depending on what it is called on.

- Example: A *Shape* class has a method `draw()`. *Circle*, *Triangle*, and *Rectangle* each have their own version of `draw()` — same name, different result.

⑧ Dynamic Binding & Message Passing

- **Dynamic binding:** The decision of which method to call is made at **runtime** (not at compile time). This supports polymorphism.
- **Message passing:** Objects communicate by sending messages to each other. A 'message' is simply a method call — specifying the object, method name, and data to send.

> **■ Golden Rule of OOP**
>
> Always design your program around **objects and the data they hold**. Ask: 'What things exist in this problem? What data do they have? What can they do?' — build classes from those answers.

3. What is Java? History & Features

Java is one of the most popular programming languages in the world. It is object-oriented, platform-independent, and designed to be simple and secure.

A Brief History

- Java was created at **Sun Microsystems** in 1991 by James Gosling and his team.
- It was first called **Oak**, then renamed to **Java** in 1995.
- Sun Microsystems was later acquired by Oracle Corporation.

> **■ Why is it called Java?**
>
> The name 'Java' comes from Java coffee — the developers loved coffee! The language was meant to be strong, widely-used, and energising — just like the drink.

Key Features of Java

Feature	What It Means
Simple	No pointers, no memory management headaches. Clean syntax.
Object-Oriented	Everything is organised into classes and objects.
Platform Independent	Write Once, Run Anywhere (WORA) — works on any OS with a JVM.
Robust	Strong error handling; many bugs are caught at compile time.
Secure	Built-in security restrictions prevent malicious code from running.
Distributed	Built-in support for networking and internet applications.
Multithreaded	Can handle multiple tasks at the same time.
Interpreted	Code is compiled to bytecode, then interpreted by the JVM.

4. How Java Works — JVM, Bytecode & Compilation

Java's magic is its **'Write Once, Run Anywhere'** ability. Here's exactly how your Java code goes from text to running program:

Phase 1	**═■ Edit**	You write Java source code in a .java file using any text editor or IDE (like IntelliJ, Eclipse, or NetBeans).
Phase 2	**■■ Compile**	You run javac (the Java compiler). It checks for errors and converts your source code into **bytecode** stored in a .class file.

Phase 3	■ **Bytecode**	Bytecode is NOT machine code. It is platform-independent instructions that no specific computer understands natively.
Phase 4	■ **Verify**	The JVM's bytecode verifier checks that the bytecodes are valid and safe before running them.
Phase 5	■■ **Execute**	The **JVM (Java Virtual Machine)** interprets the bytecode into machine code for the specific OS and hardware — and runs it.

What is the JVM?

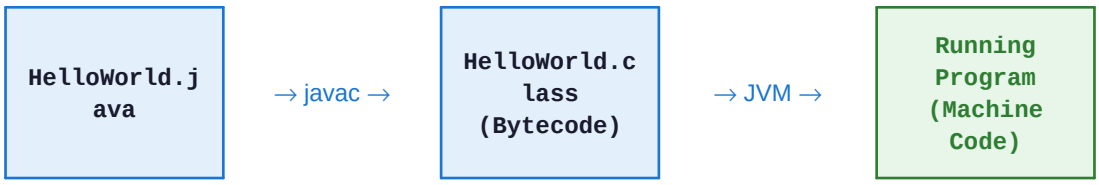
The **Java Virtual Machine (JVM)** is a software program that acts like a computer inside your computer. It reads bytecode and runs it on whatever real machine you have — Windows, Mac, Linux — without you needing to change your Java code.

> **■ The WORA Principle**
>
> Because every computer has its own JVM, the same .class bytecode file can run on any machine. This is why Java is called **platform-independent**. You compile once, run everywhere.

What is Bytecode?

- Not human-readable and not machine code — it's an intermediate format.
- Stored in `.class` files.
- Platform-independent — no dependency on any specific CPU or OS.
- The JVM translates bytecode → real machine code at runtime.

The Full Flow — At a Glance



5. Your First Java Program — Step by Step

Let's write a simple Java program and understand every single line.

```
// Java
// This is a comment — ignored by the compiler
public class Welcome {
// main() is where Java starts running your program
public static void main(String[] args) {
System.out.println("Hello, Java!"); // prints + new line
System.out.print("I like "); // prints, no new line
System.out.println("programming.");
String dept = "CSE";
System.out.println("We study in " + dept); // concatenation
}
}
```

Output:

```
// Java
Hello, Java!
I like programming.
We study in CSE
```

Breaking Down Every Part

Code Part	What It Means
public class Welcome	Declares a class named Welcome. In Java, everything lives inside a class. The file must be saved as Welcome.java.
public static void main(String[] args)	The entry point. JVM looks for this exact signature to start. public = JVM can access it. static = no object needed. void = returns nothing.
System.out.println()	Prints text to the screen and moves to a new line.
System.out.print()	Prints text but stays on the same line.
String dept = "CSE";	Declares a variable of type String (text). Variables store data.
// comment	Single-line comment. Ignored by compiler. Used to explain code.
{ }	Curly braces define blocks — where a class or method starts and ends.

■■ Class = Blueprint, Object = Building

> The class **Welcome** is like an architectural blueprint. When Java runs it, it creates an instance (object) of that blueprint. The **main()** method is the front door — Java always enters through there.

A More Complete Example — Classes & Objects

Here is a program that creates a class with data and methods, then uses it:

```
// Java
public class A {
private int a; // data field – hidden from outside
public A() { // constructor – runs when object is created
this.a = 0;
}
public void setA(int a) { // setter – lets outside set the value
this.a = a;
}
public int getA() { // getter – lets outside read the value
return this.a;
}
public static void main(String[] args) {
A ob = new A(); // create object using 'new'
ob.setA(10); // call method using dot operator (.)
System.out.println(ob.getA()); // Output: 10
}
}
```

6. Java Editions & Development Environment

Java comes in three main editions, each designed for a different purpose:

Edition	Purpose
J2SE — Standard Edition	Desktop and networking applications. The foundation — what you learn first.
J2EE — Enterprise Edition	Large-scale, distributed, and web-based applications. Used in big company systems.
J2ME — Micro Edition	Applications for small, memory-limited devices like older mobile phones and embedded systems.

Setting Up Your Development Environment

To write and run Java programs on your computer, you need:

- **JDK (Java Development Kit)** — includes the compiler (javac) and JVM. Download from oracle.com/java.

- **A text editor or IDE** — popular choices: IntelliJ IDEA, Eclipse, NetBeans, or even VS Code.

Running Java from the Command Line

```
// Java
// Step 1 — Compile your .java file
javac Welcome.java
// Step 2 — Run the compiled .class file (no extension!)
java Welcome
// Output:
Hello, Java!
```

■ Important Rules

① The file name MUST match the public class name exactly (case-sensitive). If your class is named **Welcome**, the file must be **Welcome.java**. ② A Java source file can have multiple classes, but only ONE can be public. ③ When running with the java command, do NOT include .class — just the class name.

7. OOP Advantages & Disadvantages

OOP is powerful, but like every approach, it comes with trade-offs.

Advantages of OOP ■	Disadvantages of OOP ■
• **Modularity** — code is organised into self-contained objects. • **Code reuse** — inheritance lets you reuse existing code. • **Easy maintenance** — change one object without breaking others. • **Data security** — encapsulation hides sensitive data. • **Scalability** — complex systems are easier to build and extend. • **Real-world modelling** — maps naturally to real problems.	• **More code** — OOP programs are usually longer than procedural ones. • **Slower** — extra layers (objects, method calls) add overhead. • **Not always suitable** — simple scripts don't need classes. • **Harder to learn** — beginners need time to think in objects. • **Polymorphism overhead** — dynamic binding costs processing time.

8. Common Mistakes to Avoid

These are the mistakes every beginner makes. Learn them now and save hours of frustration.

■ Common Mistake	■ The Fix
File saved as welcome.java but class is Welcome — Java is case-sensitive!	Always name the file exactly the same as the public class: Welcome.java
■ Common Mistake	■ The Fix
Running: java Welcome.class — including the .class extension causes an error.	Run: java Welcome — no extension. Java finds the .class file automatically.
■ Common Mistake	■ The Fix
Forgetting the main() signature: writing main(String args) instead of main(String[] args).	Always write: public static void main(String[] args) — exact signature required.
■ Common Mistake	■ The Fix
Making all fields public — any code can change them directly, bypassing validation.	Use private fields + public getter/setter methods to control access safely.

■ Common Mistake	■ The Fix
Confusing a class with an object — thinking Animal animal; creates an animal.	Animal animal = new Animal(); — you MUST use 'new' to actually create an object.

9. Quick Reference Card

Question	Answer
Who created Java?	James Gosling at Sun Microsystems (1991), renamed Java in 1995.
What does WORA mean?	Write Once, Run Anywhere — Java code runs on any OS with a JVM.
What is .java vs .class?	.java = source code you write. .class = bytecode compiler produces.
What command compiles Java?	javac FileName.java
What command runs Java?	java ClassName (no .class extension)
What is the JVM?	Java Virtual Machine — reads bytecode and runs it on any platform.
What is bytecode?	Platform-independent instructions stored in .class files.
What is a class?	A blueprint/template. Defines what objects look like and can do.
What is an object?	A real instance of a class, created with the new keyword.
What is encapsulation?	Bundling data + methods together; hiding internal details.
What is inheritance?	A subclass acquiring properties/methods of a superclass.
What is polymorphism?	Same method name, different behaviour depending on the object.
What is main()?	The entry point. JVM starts every Java program from main().
OOP vs Procedural?	OOP = data-centred, objects, bottom-up. Procedural = function-centred, top-down.

Summary compiled from lecture slides (OOP Concepts — Rafat Ara, Assistant Professor, CSE, UITS) and (Introduction to Java — Syeda Ajbina Nusrat, Lecturer, CSE, UITS) · Oracle Java Documentation